# Trees, Graphs, Sorting and Searching

Data Structures and Algorithms - Units 2-3 - Non-linear structures and algorithms

## Binary trees and BSTs

A binary tree node has at most two children. In a binary search tree (BST) every key in the left subtree is smaller and every key in the right subtree is larger, so search, insert and delete take O(h) where h is the height - O(log n) when balanced, O(n) when skewed.

- Inorder (L, root, R) of a BST visits keys in sorted order.
- Preorder (root, L, R) is used to copy a tree; postorder (L, R, root) to delete one.
- Level order uses a queue.
- AVL and red-black trees rotate to stay balanced.

## Heaps

A binary heap is a complete binary tree stored in an array where each parent is <= its children (min-heap). Insert and extract-min cost O(log n); building a heap from n items costs O(n). Heaps power priority queues and heap sort.

## Graphs

A graph G = (V, E) can be stored as an adjacency matrix (O(V^2) space, O(1) edge check) or adjacency lists (O(V+E) space).

```
from collections import deque
def bfs(g, s):
    seen, q = {s}, deque([s])
    while q:
        u = q.popleft()
        for v in g[u]:
            if v not in seen:
                seen.add(v); q.append(v)
    return seen
```

- BFS finds shortest paths in unweighted graphs; DFS is used for cycle detection and topological sort.
- Dijkstra finds shortest paths with non-negative weights in O((V+E) log V) using a heap.
- Kruskal and Prim build minimum spanning trees.

## Sorting

- Bubble, selection, insertion: O(n^2). Insertion sort is fast on nearly sorted data.
- Merge sort: O(n log n) always, stable, needs O(n) extra space.
- Quick sort: O(n log n) average, O(n^2) worst case, in-place.
- Heap sort: O(n log n), in-place, not stable.